

Phase 5

Controlling an Actor via LSTM-NN-Tim Dorozynski

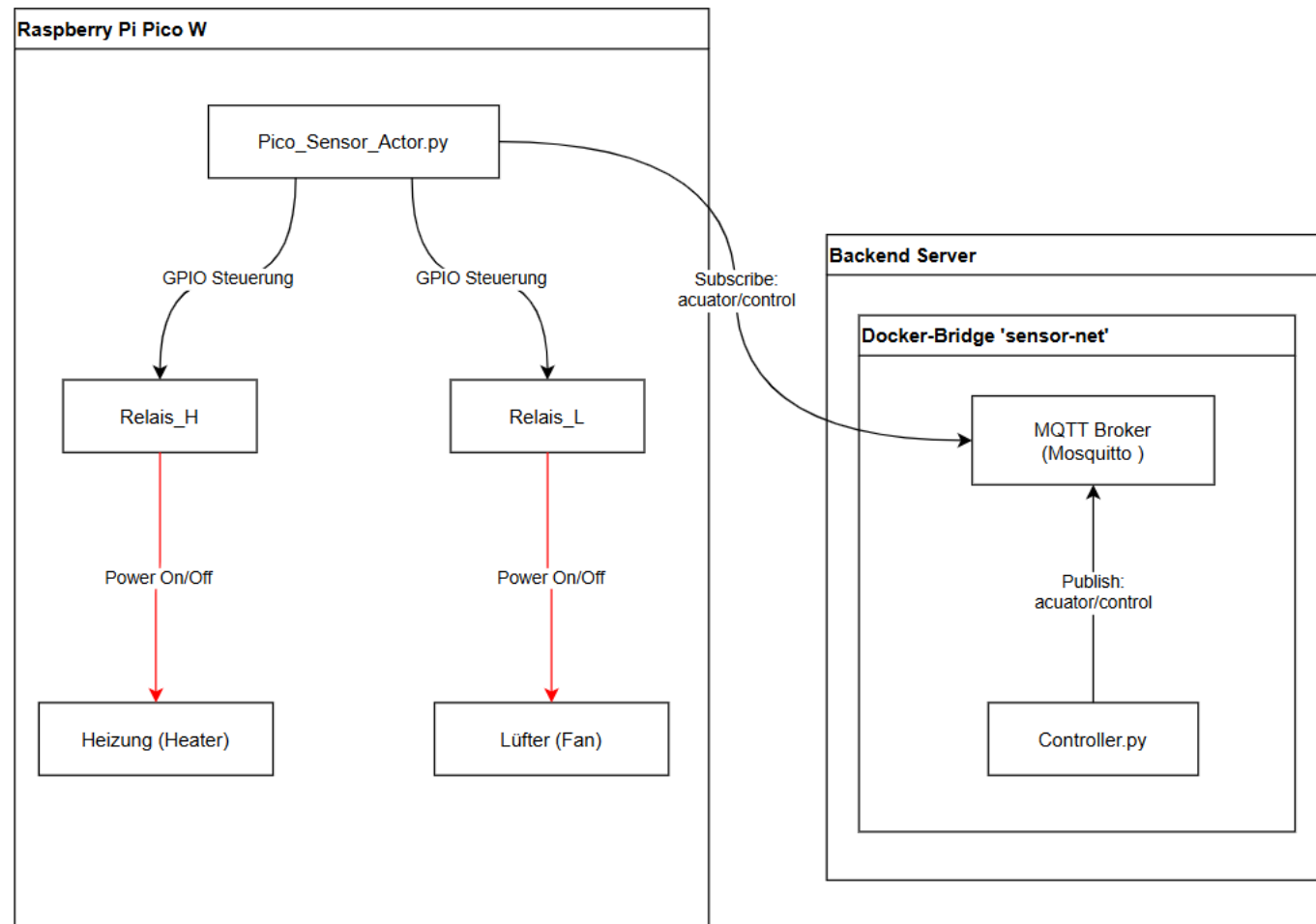
Inhaltsverzeichnis

- Das Problem
- System Architektur
- Long Short-Term Memory
- LSTM-Training des Modells
- LSTM-Vorhersage
- Verdrahtung der Aktoren
- Fazit

Das Problem

- Temperatur im Klebestand muss zwischen 19°C und 21°C bleiben.
- Reaktive Steuerung (z. B. "Heizung an, wenn $<19^{\circ}\text{C}$ ") ist zu langsam → Temperatur schwankt stark.
- Lösung: Vorhersage der Temperatur im Voraus → Aktoren schalten, bevor der Grenzwert erreicht wird

System Architektur



Long Short-Term Memory

```
5  from tensorflow.keras.models import Sequential
6  from tensorflow.keras.layers import LSTM, Dense, Dropout, Input
7  from tensorflow.keras.optimizers import Adam
8
9  SEQ_LEN = 10
10 FEATURES = 2
11
12 input_shape = (SEQ_LEN, FEATURES)
13
14 # Erstellt das LSTM-Modell mit zwei LSTM-Schichten und einer Ausgabeschicht.
15 # Dieses Modell wird für Vorhersagen verwendet.
16 def build_model():
17     model = Sequential([
18         Input(shape=input_shape),
19         LSTM(64, return_sequences=True),
20         Dropout(0.2),
21         LSTM(64),
22         Dropout(0.2),
23         Dense(1),
24     ])
25     model.compile(optimizer=Adam(learning_rate=0.001), loss="mse")
26     return model
```

- Wie verwenden TensorFlow
- Dual-Stack LSTM für Zeitreihenanalyse
- Eingabe: 10-Zeitschritte mit 2 Features
- Dropout (20%) verhindert Overfitting
- Adam-Optimierer

LSTM-Training des Modells

- Das Netz wurde mit Sensordaten Trainiert
- Sie Sensordaten werden aus einer CSV geladen
- Damit werden die weights für die KI erstellt

```
def load_csv_data(csv_file, temp_col=0, humidity_col=1, skip_header=False):  
    if not os.path.exists(csv_file):  
        raise FileNotFoundError(f"CSV-Datei nicht gefunden: {csv_file}")  
    rows = []  
    with open(csv_file, newline="", encoding="utf-8", errors="replace") as f:  
        reader = csv.reader(f)  
        if skip_header:  
            next(reader, None)  
        for line_no, row in enumerate(reader, start=2 if skip_header else 1):  
            if len(row) <= max(temp_col, humidity_col):  
                if len(row) == 0 or all(not cell.strip() for cell in row):  
                    continue  
                raise ValueError(f"Zeile {line_no} hat zu wenige Spalten: {len(row)}")  
            try:  
                temp = float(row[temp_col])  
                humidity = float(row[humidity_col])  
            except ValueError:  
                continue  
            rows.append((temp, humidity))  
    if not rows:  
        raise ValueError("Keine verwertbaren Messdaten in der CSV-Datei gefunden.")  
    return np.array(rows, dtype=np.float32)
```

LSTM-Vorhersage

```
def forecast_future(minutes=30, alpha=0.2):
    """Rekursive Vorhersage für die nächsten Minuten."""

    if len(buffer) < SEQ_LEN:
        return None

    future_predictions = []
    seq = np.array(buffer, dtype=np.float32)
    seq = np.expand_dims(seq, axis=0)      # (1, SEQ_LEN, FEATURES)

    last_humidity = float(seq[0, -1, 1])  # Humidity konstant halten

    for _ in range(minutes):
        pred = model.predict(seq, verbose=0)
        # Vorhersagewert glätten
        next_temp = (1 - alpha) * float(pred[0, 0]) \
                    + alpha * float(seq[0, -1, 0])
        future_predictions.append(next_temp)

        # Alte Sequenz ab Index 1 mit neuem Wert entlang Zeitachse verbinden
        next_step = np.array([[next_temp, last_humidity]], dtype=np.float32)
        seq = np.concatenate([seq[:, 1:, :], next_step], axis=1)

    return future_predictions
```

```
def predict_next_value(temperature, humidity):

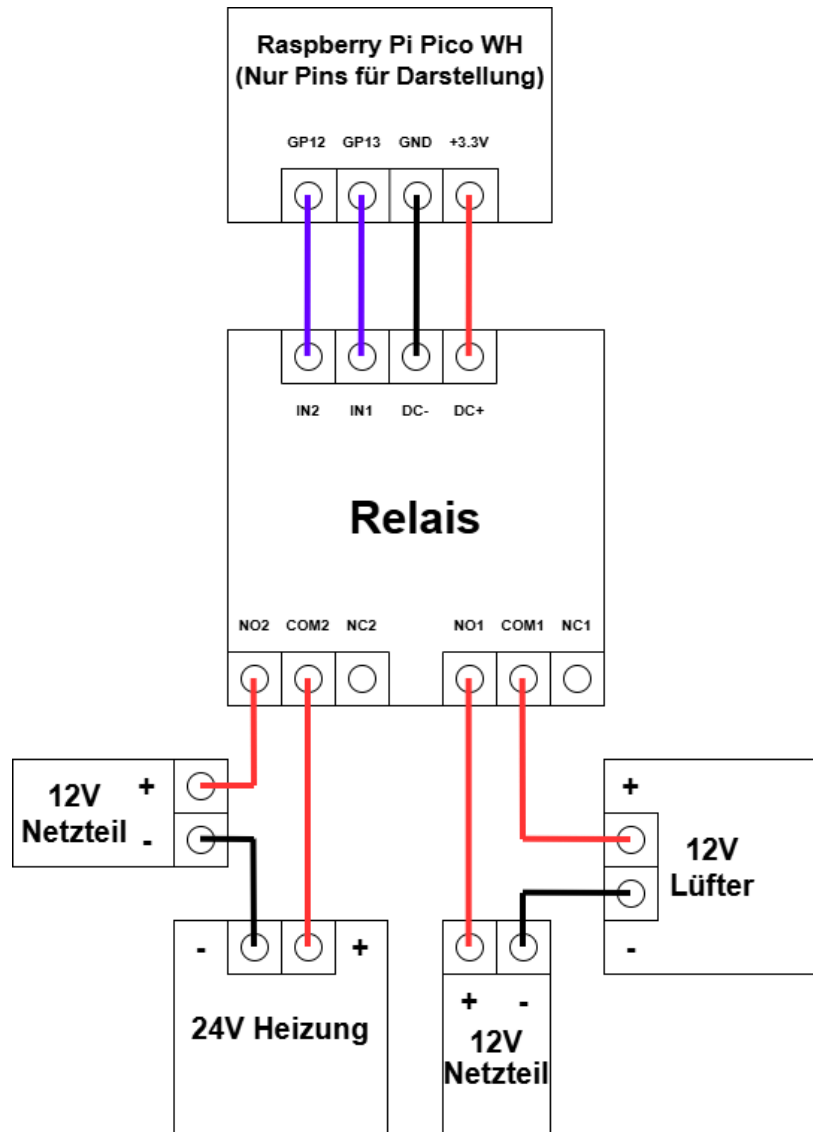
    current_features = [temperature, humidity]
    buffer.append(current_features)

    # only predict when enough data in the buffer
    if len(buffer) < SEQ_LEN:
        return None

    # reshape buffer to (1, SEQ_LEN, FEATURES) for LSTM input
    input_data = np.array(buffer).reshape((1, SEQ_LEN, FEATURES))

    # predict next value
    prediction = model.predict(input_data, verbose=0)

    return float(prediction[0][0])
```



Verdrahtung der Aktoren

- Raspberry Pi Pico WH
- Relais-Modul
- Eingänge: IN1, IN2 (Steuersignale)
- Stromversorgung: DC+, DC- (3,3V vom Pico)
- Ausgänge: NO1/COM1/NC1 (Lüfter), NO2/COM2/NC2 (Heizung)
- Aktoren:
Lüfter: 12V
Heizung: 24V

Fazit

- KI-Probleme: Vorhersage zu ungenau ($\pm 0,01^{\circ}\text{C}$)
→ Falsche Aktorsteuerung (z. B. 25°C → Vorhersage $20,96^{\circ}\text{C}$)
- Stromversorgung kritisch (500mA)
→ Instabilität durch LCD/Relais/Sensor
- MQTT-Verzögerungen (funktioniert, aber an der Grenze)
- IPs hatten sich geändert

**Vielen Dank
für Eure Aufmerksamkeit!**

Gibt es noch Fragen?